

### Esercizio A1

Le tre equazioni di ricorrenza e i relativi parametri  $\alpha$ ,  $\beta$  sono riassunti nella tabella seguente. Listiamo inoltre il caso in cui ci troviamo, per quanto riguarda il Teorema delle Ricorrenze Lineari con Partizione Bilanciata.

| Funzione                        | $\alpha$ | $\beta$ | Caso                | Complessità          |
|---------------------------------|----------|---------|---------------------|----------------------|
| $T_2(n) = 4T_2(n/2) + n^{1.0}$  | 2.0      | 1.0     | Caso 1: $1.0 < 2.0$ | $\Theta(n^2)$        |
| $T_3(n) = 9T_3(n/3) + n^{1.5}$  | 2.0      | 1.5     | Caso 1: $1.5 < 2.0$ | $\Theta(n^2)$        |
| $T_4(n) = 16T_4(n/4) + n^{2.0}$ | 2.0      | 2.0     | Caso 2: $2.0 = 2.0$ | $\Theta(n^2 \log n)$ |

### Esercizio A2

È possibile interpretare l'input come un grafo orientato, dove gli  $n$  bastoncini sono i nodi e dove esiste un arco dal bastoncino  $X[i]$  al bastoncino  $Y[i]$ ,  $\forall i : 1 \leq i \leq n$ . A questo punto, è possibile rimuovere tutti i bastoncini se e solo se non esistono cicli.

---

**boolean shangai(int[] X, int[] Y, int n, int m)**

---

```

GRAPH G = Graph()
for i = 1 to n do
    G.addNode(i)
for i = 1 to m do
    G.addEdge(X[i], Y[i])
int clock = 0
int[] dt = new int[1...G.n]
int[] ft = new int[1...G.n]
for u = 1 to G.n do
    dt[u] = ft[u] = 0
for u = 1 to G.n do
    if dt[u] == 0 and hasCycle(G, u, clock, dt, ft) then
        return false
return true

```

---

Utilizzo qui la versione che si trova nei lucidi, che prende esplicitamente  $dt$ ,  $ft$  e  $clock$ , al fine di essere precisi. Come spesso avviene nel caso delle visite in profondità, non è detto che una singola visita raggiunga tutti i nodi ed è necessario farla ripartire da ogni nodo che non è già stato visitato.

La complessità della costruzione del grafo è  $O(m + n)$ , pari al costo della visita in profondità realizzata da `hasCycle()`. Il costo totale è quindi  $O(m + n)$ .

Ho visto in alcuni compiti soluzioni come la seguente:

---

**shangai(int[] X, int[] Y, int n, int m)**

---

```

GRAPH G = Graph()
for i = 1 to n do
    G.addNode(i)
for i = 1 to m do
    G.addEdge(X[i], Y[i])
return not ciclico(G)

```

---

in cui si assumeva che la funzione `ciclico(G, u)` che si trova nel libro abbia un wrapper che applica la funzione all'intero grafo. Ho ritenuto la soluzione accettabile, in quanto in pochi si sono accorti che

il problema era facilmente risolvibile e la mancanza del wrapper nel libro è una mia mancanza (lo aggiungerò ai lucidi).

### Esercizio A3

Anche questo problema è sostanzialmente un problema su grafo, ma non è necessario trasformarlo in grafo per risolverlo; complicherebbe inutilmente le cose.

L'idea è di simulare una visita BFS a partire da tutte le celle che hanno come valore 1, in modo simile a quanto fatto nell'esercizio 3 del 6/6/2011 (visto normalmente a lezione durante le esercitazioni). Si utilizza la struttura della funzione `erdos()` vista a lezione, modificata per tener conto che lavoriamo su una matrice e non su un grafo.

I vettori  $dr$  e  $dc$  rappresentano le differenze delle coordinate delle quattro possibili mosse; i valori  $nr, nc$  rappresentano le nuove coordinate una volta che vengono applicate le differenze.

La complessità è pari a  $O(n^2)$ , in quanto ogni casella entra nella coda al massimo una volta.

Nel codice, abbiamo inserito nella coda coppie di elementi, come per esempio è possibile fare in Python senza troppi problemi. Se questo non piace, è possibile semplicemente inserire ed estrarre dalla coda due valori alla volta (riga, colonna).

---

```
int grid(int[][] M, int n)
    int[] dr = [-1, 0, +1, 0]                % Mosse possibili sulle righe
    int[] dc = [0, -1, 0, +1]                % Mosse possibili sulle colonne
    int[] distance = new int[1...n][1...n]
    QUEUE Q = Queue()
    for r = 1 to n do
        for c = 1 to n do
            distance[r][c] = if(M[r][c] == 1, 0, -1)
            if M[r][c] == 1 then
                Q.enqueue(<r, c>)
    while not Q.isEmpty() do
        int, int r, c = Q.dequeue()           % Riga, colonna della cella visitata correntemente
        for i = 1 to 4 do
            nr = r + dr[i]                    % Nuova riga
            nc = c + dc[i]                    % Nuova colonna
            if 1 ≤ nr ≤ n and 1 ≤ nc ≤ n and distance[nr][nc] < 0 then
                distance[nr][nc] = distance[r][c] + 1
                if M[nr][nc] == 3 then
                    return distance[nr][nc]
                else
                    Q.enqueue(<nr, nc>)
```

---

### Esercizio B1

Ovviamente, è necessario utilizzare la tecnica backtrack. La struttura è quella di una visita in profondità, in cui però si mantiene un contatore per memorizzare il numero di nodi visitati finora.

Quando il numero di nodi visitati è pari a  $k + 1$ , il numero di archi visitati (e quindi la lunghezza del cammino) è pari a  $k$  e quindi si stampa il cammino.

Per elencare tutti i cammini possibili, evitando comunque di ripetere nodi, si utilizza l'accorgimento di marcare il nodo  $u$  come visitato all'inizio della sua visita ( $visited[u] = \mathbf{true}$ ), e rimuovere il marcatore ( $visited[u] = \mathbf{false}$ ) dopo che la visita è terminata. In questo modo potrà essere visitato di nuovo in altri cammini.